

IMPLEMENTAÇÃO DE MÁQUINAS ABSTRATAS: AFD, AUTÔMATO DE PILHA E MÁQUINA DE TURING

Amanda Pimentel - 72301260
Emilly Luiza V. Cordeiro - 72301279

Fundamentos Teóricos da Computação
Faculdade Cotemig
2026

1 Introdução

A Teoria da Computação estuda modelos matemáticos utilizados para representar e analisar processos computacionais. Entre os principais modelos estão os Autômatos Finitos Determinísticos (AFD), os Autômatos de Pilha (AP) e as Máquinas de Turing (MT), que possuem diferentes capacidades de reconhecimento e processamento de linguagens formais.

Este trabalho teve como objetivo implementar simuladores desses modelos na linguagem C#, aplicando na prática os conceitos estudados na disciplina. As implementações seguiram suas definições formais e permitiram acompanhar o processamento das cadeias de entrada.

Foram realizados testes com diferentes entradas, incluindo um AFD para linguagens regulares, dois Autômatos de Pilha para linguagens livres de contexto e duas Máquinas de Turing, uma reconhecedora e uma computadora. Os resultados permitiram observar as diferenças de poder computacional entre os modelos e compreender suas aplicações no estudo das linguagens formais e da computabilidade.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é definido formalmente pela 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

onde:

- Q é o conjunto de estados;
- Σ é o alfabeto de entrada;
- δ é a função de transição;
- q_0 é o estado inicial;
- F é o conjunto de estados finais.

O AFD implementado reconhece a linguagem:

$$L_1 = \{w \in \{a, b\}^* \mid w \text{ termina com } ab\}$$

Exemplo 1

Cadeia: bab

Símbolo	Estado
Início	q_0
b	q_0
a	q_1
b	q_2

Como $q_2 \in F$, a cadeia é aceita.

Exemplo 2

Cadeia: ba

Símbolo	Estado
Início	q_0
b	q_0
a	q_1

A computação termina em q_1 , portanto a cadeia é rejeitada.

2.2 Autômato de Pilha

Um Autômato de Pilha (AP) é uma extensão dos autômatos finitos que utiliza uma estrutura de memória do tipo pilha, permitindo o reconhecimento de linguagens livres de contexto.

Formalmente, um AP com aceitação por pilha vazia é definido pela 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$$

onde:

- Q é o conjunto de estados;
- Σ é o alfabeto de entrada;
- Γ é o alfabeto da pilha;
- δ é a função de transição;
- q_0 é o estado inicial;
- Z_0 é o símbolo inicial da pilha;
- a aceitação ocorre quando a pilha é completamente esvaziada.

Foram implementados dois APs distintos.

O primeiro reconhece a linguagem:

$$L_2 = \{a^n b^n \mid n \geq 1\}$$

utilizando um autômato determinístico que empilha um símbolo para cada ocorrência de a e desempilha um símbolo para cada ocorrência de b .

O segundo reconhece a linguagem:

$$L_3 = \{w \in \{a, b\}^* \mid w = w^R, |w| \geq 1\}$$

correspondente aos palíndromos sobre o alfabeto $\{a, b\}$.

Para o reconhecimento dos palíndromos foi utilizado um AP não determinístico capaz de adivinhar o ponto médio da cadeia por meio de movimentos- ε , permitindo o tratamento tanto de cadeias de comprimento par quanto ímpar.

A implementação registra todas as configurações instantâneas visitadas durante a computação, exibindo estado corrente, entrada restante, conteúdo da pilha e transição aplicada.

2.3 Máquina de Turing

A Máquina de Turing (MT) é um modelo computacional capaz de simular qualquer algoritmo executável.

Formalmente, uma MT é definida pela 7-tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

onde:

- Q é o conjunto de estados;
- Σ é o alfabeto de entrada;
- Γ é o alfabeto da fita;
- δ é a função de transição;
- q_0 é o estado inicial;
- q_{accept} é o estado de aceitação;
- q_{reject} é o estado de rejeição.

Foram implementadas duas Máquinas de Turing.

A primeira reconhece a linguagem:

$$L_4 = \{a^n b^n c^n \mid n \geq 1\}$$

utilizando uma estratégia de marcação sucessiva. A cada ciclo, um símbolo a , um símbolo b e um símbolo c são marcados e a máquina retorna ao início da fita para repetir o procedimento.

A segunda máquina implementa uma computação sobre números naturais representados em notação unária:

$$f(n) = n + 1$$

Nesse caso, a máquina percorre toda a sequência de símbolos 1 até encontrar o branco da fita, escreve um novo símbolo 1 e encerra a computação.

Durante a execução são exibidas todas as configurações instantâneas da fita, incluindo estado atual, posição do cabeçote e transição aplicada.

3 Descrição da Implementação

3.1 Implementação do AFD

A implementação seguiu diretamente a definição formal da 5-tupla.

As estruturas utilizadas foram:

- Conjunto de estados (Q);
- Alfabeto (Σ);
- Função de transição (δ);
- Estado inicial (q_0);
- Conjunto de estados finais (F).

A função de transição foi implementada utilizando um *Dictionary*, permitindo acesso eficiente às transições.

O método `Aceitar()` percorre a cadeia símbolo por símbolo atualizando o estado atual conforme a função δ .

Além disso, foi implementado o carregamento dinâmico de AFDs a partir de arquivos JSON.

3.2 Implementação do AP

A implementação dos Autômatos de Pilha seguiu rigorosamente a definição formal da 7-tupla.

Os conjuntos de estados, alfabeto de entrada e alfabeto da pilha foram armazenados utilizando estruturas `HashSet`.

A função de transição foi implementada utilizando:

```
Dictionary<(estado, entrada, topoPilha),  
           List<(novoEstado, empilhar)>>
```

permitindo representar naturalmente o não determinismo.

Os movimentos- ϵ foram representados internamente pelo caractere nulo ($\backslash 0$).

Para o AP de palíndromos foi utilizado um algoritmo de busca em profundidade (Depth First Search – DFS) com retrocesso (backtracking), permitindo explorar diferentes caminhos computacionais até encontrar uma computação aceitante.

As configurações instantâneas são armazenadas em estruturas específicas contendo:

- número do passo;
- estado atual;
- entrada restante;
- conteúdo da pilha;
- transição aplicada.

A aceitação ocorre exclusivamente quando toda a entrada é consumida e a pilha se torna vazia.

3.3 Implementação da Máquina de Turing

A fita da Máquina de Turing foi implementada utilizando a estrutura:

```
Dictionary<int, char>
```

permitindo expansão dinâmica para a esquerda e para a direita.

As transições foram armazenadas em uma tabela do tipo:

```
Dictionary<(estado, simbolo),  
           (novoEstado, novoSimbolo, direcao)>
```

onde a direção pode assumir os valores L (Left) ou R (Right).

Durante a simulação são registradas todas as configurações instantâneas da máquina, incluindo:

- estado atual;
- conteúdo da fita;
- posição do cabeçote;

- transição executada;
- quantidade total de passos.

Foi implementado ainda um mecanismo de limite máximo de passos configurável, utilizado para prevenir laços infinitos durante a execução da máquina.

4 Resultados e Testes

4.1 AFD – Linguagem L_1

Cadeia	Resultado
ab	ACEITA
aab	ACEITA
bab	ACEITA
ababab	ACEITA
ba	REJEITA
ε	REJEITA
b	REJEITA

4.2 AP – Linguagem L_2

Cadeia	Resultado
ab	ACEITA
aabb	ACEITA
aaabbb	ACEITA
aab	REJEITA
abb	REJEITA
ba	REJEITA
ε	REJEITA
abab	REJEITA

4.3 AP – Linguagem L_3

Cadeia	Resultado
a	ACEITA
aba	ACEITA
abba	ACEITA
ab	REJEITA
aab	REJEITA

4.4 MT – Linguagem L_4

Cadeia	Resultado
abc	ACEITA
aabbcc	ACEITA
aaabbbccc	ACEITA
aabbc	REJEITA
ab	REJEITA
abcabc	REJEITA
ε	REJEITA

5 Análise Comparativa

O AFD é o modelo menos poderoso entre os três estudados, sendo capaz de reconhecer apenas linguagens regulares. Seu funcionamento depende exclusivamente do estado atual, não possuindo memória auxiliar.

O AP amplia esse poder computacional ao incorporar uma pilha, permitindo reconhecer linguagens livres de contexto, como $a^n b^n$. Entretanto, continua incapaz de reconhecer linguagens que exigem múltiplas comparações independentes.

A Máquina de Turing representa o modelo mais poderoso estudado. Sua fita potencialmente infinita permite armazenamento ilimitado de informações e execução de algoritmos gerais.

Existe uma hierarquia clara entre os modelos:

$$AFD \subset AP \subset MT$$

demonstrando o aumento gradual do poder computacional.

6 Conclusão

A implementação das três máquinas abstratas permitiu compreender de forma prática os conceitos apresentados na disciplina de Fundamentos Teóricos da Computação. A tradução das definições matemáticas para código reforçou a importância da formalização no desenvolvimento de sistemas computacionais.

Os resultados obtidos evidenciaram as diferenças entre linguagens regulares, livres de contexto e linguagens reconhecidas por Máquinas de Turing. Além disso, a atividade demonstrou como estruturas de memória adicionais ampliam significativamente o poder computacional de um modelo.

O trabalho contribuiu para consolidar conhecimentos fundamentais sobre reconhecimento de linguagens formais e computabilidade, servindo como base para estudos mais avançados na

área.

7 Questões Reflexivas

7.1 Parte 1

1. Por que a linguagem L_1 é regular?

A linguagem L_1 é regular porque sua aceitação depende apenas dos dois últimos símbolos da cadeia. Um número finito de estados é suficiente para armazenar essa informação. Assim, existe um AFD capaz de reconhecer a linguagem. A expressão regular equivalente é:

$$(a|b)^*ab$$

2. Descreva formalmente a 5-tupla do AFD construído.

$$M = (Q, \Sigma, \delta, q_0, F)$$

onde:

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$F = \{q_2\}$$

O estado q_0 representa a situação inicial, q_1 indica que o último símbolo lido foi a , e q_2 indica que a cadeia termina com ab .

3. O que aconteceria se δ não fosse total?

Se a função de transição não fosse total, poderiam existir combinações de estado e símbolo sem transição definida. Isso tornaria a computação indefinida. Na implementação, símbolos inválidos e transições inexistentes resultam em rejeição da cadeia.

7.2 Parte 2

1. Por que L_2 não pode ser reconhecida por um AFD?

Pelo Lema do Bombeamento, assumindo que L_2 seja regular, a cadeia $a^p b^p$ poderia ser bombeada produzindo mais símbolos a do que b , gerando uma cadeia fora da linguagem. Isso contradiz o lema, demonstrando que L_2 não é regular.

2. Aceitação por pilha vazia e por estado final são equivalentes?

Sim. Ambos os mecanismos reconhecem exatamente as linguagens livres de contexto. Existe uma transformação formal que converte um AP de um modelo para o outro sem alterar a linguagem reconhecida.

3. Explique o papel do símbolo Z_0 .

O símbolo Z_0 atua como marcador de fundo da pilha. Ele permite identificar quando todas as informações empilhadas foram removidas e garante que a aceitação ocorra apenas quando a pilha estiver realmente vazia.

7.3 Parte 3

1. Por que L_4 não pode ser reconhecida por um AP?

A linguagem $a^n b^n c^n$ exige comparação simultânea entre três quantidades. Um AP possui apenas uma pilha e consegue comparar apenas duas regiões da entrada. A Máquina de Turing resolve esse problema utilizando memória potencialmente ilimitada.

2. Quantos estados foram necessários e qual foi a estratégia de marcação?

Foram utilizados estados responsáveis por localizar e marcar símbolos a , b e c . A estratégia consiste em substituir símbolos já processados por marcadores especiais e repetir o procedimento até que toda a fita seja validada.

3. Um computador moderno é mais poderoso que uma Máquina de Turing?

Segundo a Tese de Church-Turing, qualquer algoritmo executável por um computador moderno também pode ser executado por uma Máquina de Turing. Portanto, computadores atuais são mais rápidos e eficientes, mas não possuem maior poder computacional teórico.

8 Referências

MENEZES, P. B. Linguagens formais e autômatos. 6. ed. Porto Alegre: Bookman, 2010. v. 3. Disponível em: <https://biblioteca-a.read.garden/viewer/9788577807994/1>

HOPCROFT, J.; MOTWANI, R.; ULLMAN, J. Introdução à Teoria de Autômatos, Linguagens e Computação. Boston: Elsevier, 2002.

SOUSA, C. E. B. et al. Linguagens formais e autômatos. Porto Alegre: SAGAH, 2021. Disponível em: <https://biblioteca-a.read.garden/viewer/9786556901138/capa>